OS Supplementary Material 1: A brief introduction to Linux

Linux is the common name given to a large family of operating systems. A large variety of software may be installed and run on Linux OS, including web browsers, text editors, multimedia players, games; or more specialized software such as scientific tools, web and database servers, etc.

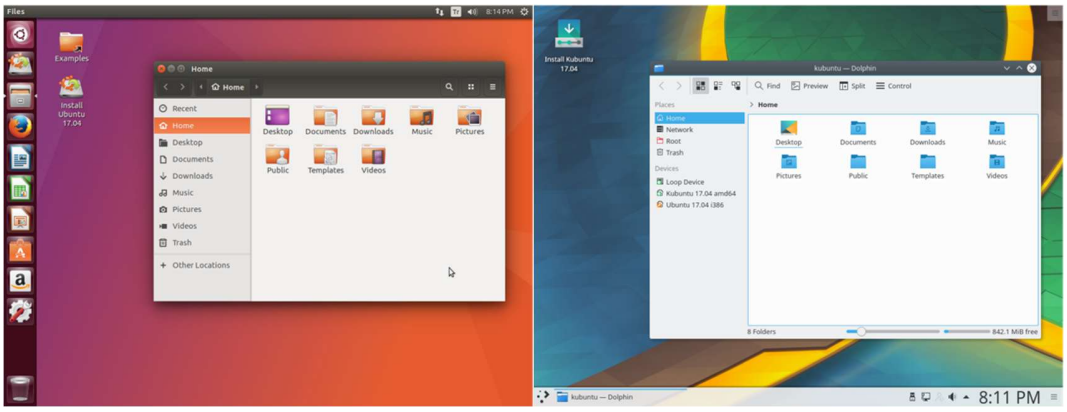
As of now, there are over 100 different Linux-based operating systems. Ubuntu, Mint, Debian, Open- SUSE, Fedora are only a few of the most popular ones. The differences between all these operating systems are usually in terms of the user interface or how the operating system handles the background tasks. For example Kubuntu is different from Ubuntu only in terms of user interface and the default set of programs bundled with them; but all the other parts (Linux kernel, system services, etc) are exactly the same and common operations can be performed in a similar manner in both operating systems.

Figure 1: Ubuntu and Kubuntu desktops

This document is based on KDE neon operating system, but you can install any Linux-based operating system on your own computers. Ubuntu is recommended for beginners because of its popularity and it has a large resource of information on the web.

1 Introduction to Command Line Interface

The aim of this manual is to get you acquainted with the Command Line Interface of Linux. The Command Line Interface (CLI) is a powerful tool for performing various tasks on the system. Basically you enter a command from keyboard (which tells the computer to do something) and the computer executes your command, maybe interacting with you by displaying output or asking for additional input. When the command finishes executing, the system waits for another command from the user.



1

Command Line Interface can be accessed by “Console” or “Terminal” application. On KDE, you may open this application by opening the application menu on the bottom-left corner of the screen and searching for “Konsole”. When you see the application, click on its icon.

The black screen you now see is the Command Line Interface. In it, you should see a single line of text. This is the “prompt line”; you may enter your commands when you see it. If command prompt does not appear after executing a command for any reason (program stuck in a loop, etc), you may hit Ctrl-C from keyboard to break the current program and forcefully return back to the prompt.

1.1 Basics

For example, if you type date command: “date” from keyboard and hit Enter key, you will see the data.

You should see that the computer has responded to your command by displaying the current day and time. Try date command a few seconds later again and you should see that the computer responds with a different output.

Try the following commands one by one:

• whoami - displays the username of the current user.
• cal - displays a calendar.
• uptime - displays how long the system has been running.

• echo - tells the computer to repeat a string:

– echo Hello World
– echo $USER - What do you see on screen? Why?
– echo Hello $USER - Combination of previous two examples.
– echo Two plus five is $((2+5)) - The CLI may help you with mathematics too.

1.2 Working With Files

Echo command can also help you with creating text files. Enter the following commands:

```
echo file fopen stdio.h > list.txt
echo int pipe unistd.h >> list.txt
echo void malloc stdlib.h >> list.txt
echo int dup unistd.h >> list.txt
echo void pthread_exit pthread.h >> list.txt
echo char fgets stdio.h >> list.txt
echo int chdir unistd.h >> list.txt
```

Remember that when you used echo command previously, the output was written to the screen. But if you use “>” or “>>” signs, the output will instead be written to a specified file (list.txt in this case). Note that using “>” will clear everything in the file and start from beginning, while “>>” appends to the end of file without clearing previous contents. This is analogous to the difference between opening files with “w” or “a” modes with fopen() function in C programming language.

Now that you have created a file, lets see if it is really there. Type “ls” command. This command lists available files and directories. Along with some directories, you should see your newly created file too. If you want to view contents of this file, you must execute “cat list.txt”:

wc list.txt - Count words, lines and characters in list.txt

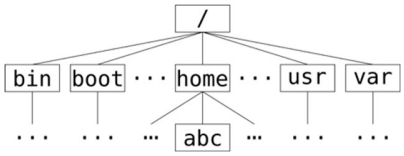
grep stdio list.txt - Search for the word ‘‘stdio’’ in ‘‘list.txt’’ and display the lines in which this word appears

sort list.txt - alphanumerically sorts lines in list.txt file and display results on screen
sort list.txt > sorted.txt - sorts list.txt into sorted.txt file
cp list.txt os.txt - make a new copy of list.txt named as os.txt
mv os.txt fun.txt - rename os.txt into fun.txt

1.3 Working With Directories

In Linux based operating systems, files and directories are stored in an hierarchy as shown below:

When you first open Console, you usually start in your home directory, which resides in "/home/" directory (e.g. /home/abc/, /home/student/, etc). All files and directories owned by your account are stored in your home directory by default.



Execute the following commands one by one and understand how they work:

pwd	- to view in which directory you are working now
pwd ls	- to view which files and directories are available in your current directory
cd Desktop	- change current directory to Desktop
mkdir lab1	- make a new directory named lab1
ls	- to view which files and directories are available in your current directory
cd lab1 pwd	- change current directory to lab1
cd ..	- go back to parent directory

You can move your files and directories into other directories. For example, execute the following:

- mv fun.txt Desktop/lab1/
- mv sorted.txt Desktop/lab1/

1.4 Removing Files and Directories

Go into your lab1 directory by properly using pwd and cd commands. Then execute the following commands:

- rm fun.txt - removes fun.txt file.
- rm sorted.txt - removes sorted.txt file.
- cd .. - go to parent directory.
- rmdir lab1 - remove lab1 directory.

Note that rmdir requires the target directory to be completely empty before removing. Be careful! Using rm and rmdir commands will IMMEDIATELY delete files and directories; you can not recover them back.

1.5 Getting Help

You may get a detailed help document for any command with man utility. For example, try "man ls" now. You can scroll up and down using arrow keys. After reading the document, you may go back to command prompt by hitting q key.

You may get detailed documentation about C programming functions with man utility as well. Try "man 3 printf", "man 3 scanf", "man 3 strcpy", "man 3 islower", etc.

1.6 Permissions

Executing “ls -l” will generate a detailed list of information about your files and directories:

```
permission  owner  group  size  date      filename
drwxr-xr-x 5 student student 20480 May 21 13:47 Desktop
-rw-r----- 1 student student 141   Feb 12 10:50 list.txt
```

The permissions are divided into 4 parts:

- The first position specifies what type of file this is. - means regular file, d means directory, etc. This part will be explained in details in Laboratory Chapter 7 - File Systems.
- The next three parts are divided into three sets of people: user, group, others. Each group gets three kinds of permission: read, write, execute. For example, lets focus on list.txt:

  user group others 421 421 421 rw- r-- --- 640

- “Access Mode” of this file is 640. It is calculated as “4+2+0, 4+0+0, 0+0+0”.
- The first three bits specify the permissions for the user who owns the file. In this example, the owning user can read and modify this file.
- The next trio of bits are related to other people in the same group as the owner. In this example, group members may read this file but not modify it.
- The last trio of bits are for people outside the owner’s group. In this example, they can neither read, nor modify this file.

You can alter access modes of files at any time. Execute the following commands and use “ls -l” to observe what has changed after each chmod command:

```
chmod 400 list.txt      - set read for owner and nothing else.

chmod 644 list.txt      - set read and write for owner, read for group and others.

chmod 000 list.txt      - remove all access to this file.

chmod +r list.txt       - give read access for all users.

chmod u+w list.txt      - give write access only to owner.

chmod o-r list.txt      - remove read access from others.
```

1.7 Command Line Scripts

Instead of entering commands one by one, you may store them in script files and execute them at once at any time later. Open Kate or default editor application from application menu and type the following:

```
#!/bin/bash

# this is a simple script for greeting the user
# and displaying working directory and date
        echo "Hello $USER! You are now in:"
        pwd
```

```
echo "The date and time is:"
date
```

Save this file as “hello.sh” in your Desktop. Now open a new Console and execute the following:

```
cd Desktop          - go into Desktop directory
chmod +x hello.sh   - give execution permission to hello.sh file
./hello.sh          - execute your script
```

See what output you have after execution of hello.sh.

You may define variables in your scripts, for example:-

```
#!/bin/bash

COURSE="CENG 382"
echo "My favorite course is $COURSE"
```

When executed you get following output:

```
#!/bin/bash

COURSE="CMPE 382"
echo "My favorite course is $COURSE"
```

You may also perform more complex tasks with loops, if statements in script files, etc. For example, the following script lists all files in the current directory and lists files which have their names starting with “cmpe” but not “cmpe2” (cmpelist.sh):

```
#!/bin/bash

for i in *; do
   if [[ $i == cmpe* && $i != cmpe2* ]]; then echo $i;
fi
done
```

1.8 Compiling C / C++ Programs

You may develop programs using C programming language (and MANY more!) in Linux too. As a demonstration, open Kate from application menu and write a simple hello world program in C. Save your file as “hello.c” and type the following in a new Console window:

• gcc hello.c -o helloworld - Compile hello.c source code into helloworld program file

• ./helloworld - execute the program

The gcc command translates your C source file into executable machine code. If you list files after executing gcc commmand, you should see two different files:

```
-rw-rw-r--  1 student student     65 Sep 29 12:56 hello.c
```

-rwxrwxr-x 1 student student 8600 Sep 29 12:58 helloworld

Notice that the new file, helloworld, has execution permissions.

Do note that in order to compile C++ programs, your source file must have either “cc” or “cpp” extension and you must use “g++” instead of “gcc”.

Sometimes you may make mistakes in your programs. In such cases, gcc may display either a warning or an error message:

If you encounter a warning message, that means your code has compiled, but may not work properly (e.g. runtime errors). If you encounter an error message, that means your code has not compiled. It is recommended to fix warnings in most cases, but fixing errors is compulsory.

1.8.1 Compiling Programs From Multiple Source Files

Sometimes you may need to compile programs that has its source code spanning across multiple source files. For example Figure 8 shows a portion of Mozilla Firefox source:

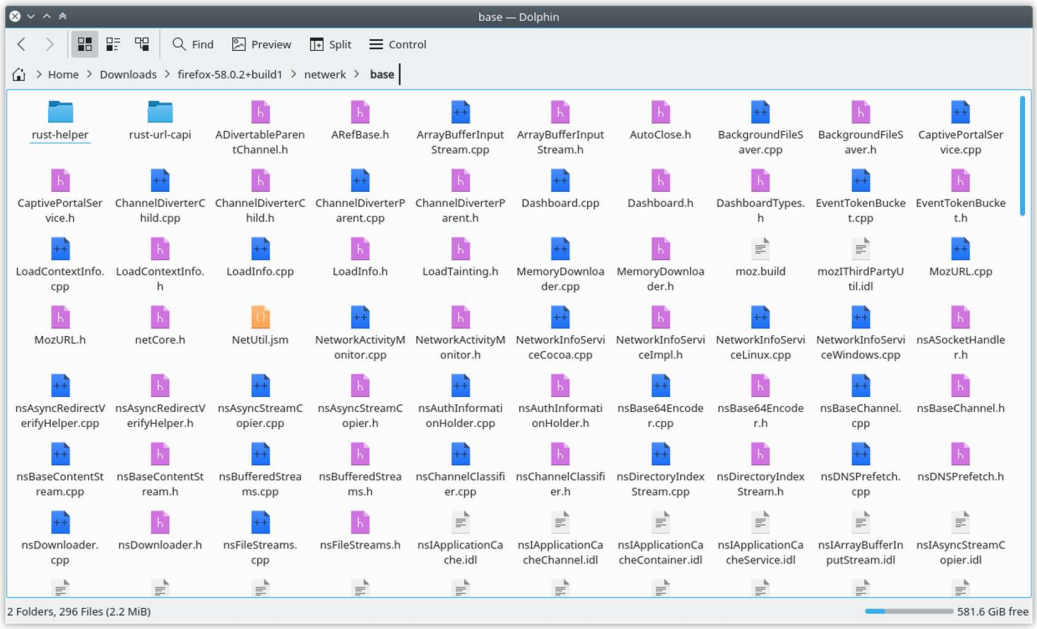


Figure 8: Multiple Source Codes

In order to compile more than one source files into one executable, you simply append every source file to your gcc line:

gcc source1.c source2.c ... -o program

1.9 Command Line Arguments in C/C++ Programs

The keywords that follow the main program names (ls, sort, gcc) in the program execution examples are named Command Line Arguments. They are used to input data to programs before running. All programs written in C / C++ languages can parse such command line arguments. In order to do this, you must provide two special variables as parameters to main function: one integer and one string array. An example program is given below (say arguments.c):

```
#include <stdio.h>
int main(int argc, char** argv) {
  int i;
  for (i = 0; i < argc; ++i) {
    printf("argv[%d] = %s\n", i, argv[i]);}
return 0; }
```

Compile this program and execute it with as many command line arguments as you want:

```
./arguments
./arguments hello world
./arguments cmpe382 "operating systems"
./arguments 2 times 2 is 4
```

What output do you see?

One important thing to pay attention is that all arguments are string variables, no exceptions. Aside from strings, all integer and floating point numbers entered as command line arguments will still be strings, therefore you must convert the corresponding strings to integers or floats by using functions such as atoi() or atof(), if it is so required.